

CS285 LECTURE 8 • ZERO-TO-HERO FIELD MANUAL

Continuous Q-Learning & Soft Actor-Critic (SAC)

From Scratch to Mastery: Conquering Continuous Robot Action Spaces, Overcoming the Deadly Triad, and Unleashing Entropy-Driven Curiosity

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab)

Focus: Continuous $\max Q$, Clipped Double-Q, Target Networks, & MaxEnt RL

1. Why Classic Q-Learning Fails on Robots (The Menu vs The Beach)

In 2015, DeepMind shocked the world with **DQN** (Deep Q-Networks), beating humans at Atari 2600 video games. Naturally, roboticists rushed to use DQN on robotic arms. **It failed completely.** Why?

THE RESTAURANT MENU VS THE INFINITE BEACH ANALOGY

In Q-learning, the optimal decision rule is: $a^* = \arg \max_a Q(s, a)$.

- **In Atari Games (The Restaurant Menu):** There are only 4 controller buttons (Up, Down, Left, Right). Finding the $\max$ is like picking dinner from a 4-item menu. You evaluate all 4, find that Pizza has score 9.5, and order Pizza. Easy!
- **On a Robotic Arm (The Infinite Beach):** Actions are continuous 6-dimensional vectors of real numbers (e.g. torques $\in [-10.0, +10.0]$ Nm). Finding the highest Q -value is like being dropped on an infinitely large, foggy beach with rolling sand dunes, and being told to find the single highest grain of sand in under **1 millisecond!** You cannot check infinite grains of sand in real time!

The Modern Fix (Actor Maximizer): Instead of searching for the highest grain of sand by brute force, train a dedicated **Actor network** $\pi_\theta(s)$ that acts as a compass, outputting the exact coordinates of the highest peak!

ATARI: 4 DISCRETE CHOICES

Up: 4.2 Down: 2.1 **Right: 9.8 (MAX!)**
Simple

`torch.argmax()` across 4 numbers vs. ROBOT: INFINITE CONTINUOUS SURFACE True Peak a^* Actor network $\pi_\theta(s)$ climbs directly to the peak!

2. Soft Actor-Critic (SAC) & Maximum Entropy Exploration

THE CURIOUS TOURIST IN PARIS ANALOGY

Imagine you visit Paris for two weeks:

- **Standard RL (Reward Only):** On your first morning, you find a bakery with decent croissants (Reward = +5). You decide this is safe and eat croissants at this exact bakery every single morning for 14 days. You never try baguettes, macarons, or escargot. You get stuck in a boring, mediocre routine.
- **Maximum Entropy RL (Reward + Entropy):** We pay you a bonus for being **curious and unpredictable!** You get points for good food, PLUS extra points for trying new alleys and testing diverse bakeries.

Why this is mandatory for robot soft tomato slicing: If a robot only cares about reward, it gets terrified of crushing penalties and freezes the blade 1 mm above the skin! The **Entropy Bonus** pays the robot to keep trying different blade angles, sawing speeds, and contact compliance until it discovers clean puncture mechanics!

2.1 The Soft Bellman Objective (Parameter Anatomy)

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t)} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \quad \text{where} \quad \mathcal{H}(\pi) = \mathbb{E}_{a \sim \pi} [-\log \pi(a | s_t)]$$

Parameter	Formal Name	Plain English Meaning	Example Value	Tuning Impact
$r(s_t, a_t)$	Physical Task Reward	Points earned for cutting the tomato cleanly without crushing.	+3.5 points	Primary task objective.
α	Entropy Temperature	The Curiosity Dial: How much does the robot prioritize exploration versus exploitation?	$\alpha = 0.2$ (or auto-tuned)	If $\alpha \rightarrow 0$, standard RL (freezes); if $\alpha \rightarrow \infty$, pure chaotic noise.
$\mathcal{H}(\pi)$	Shannon Entropy	A mathematical measure of how wide, random, and diverse the action distribution is.	1.5 nats	High entropy = wide exploration; Low entropy = laser focus.



PLAIN ENGLISH TRANSLATION OF THE SOFT RL OBJECTIVE

"Do your job as best as possible (maximize reward), but keep your actions as random and diverse as possible (maximize entropy) so you never get stuck in a timid local rut!"

3. The Three Practical Super-Weapons of SAC

Richard Sutton proved that combining *Function Approximation* + *Bootstrapping* + *Off-Policy Learning* creates the **Deadly Triad**, causing value functions to diverge to infinity. SAC overcomes the Deadly Triad using three engineering breakthroughs:

3.1 Super-Weapon 1: Experience Replay (The Photo Album)

Instead of discarding transitions immediately, SAC saves **1,000,000 past transitions** (s_t, a_t, r_t, s_{t+1}) into a circular buffer. During training, it samples random mini-batches (e.g. 256 transitions). This breaks temporal correlation: the network doesn't just learn from what happened 2 milliseconds ago; it remembers mistakes made 30 minutes ago!

3.2 Super-Weapon 2: Polyak Target Networks (The Patient Teacher)

🎯 THE MOVING TARGET ANALOGY

If you train a neural network using its own predictions as the target $(y = r + \gamma Q(s', a'))$, the target shifts every single gradient update. It is like trying to shoot a bullseye that vibrates frantically.

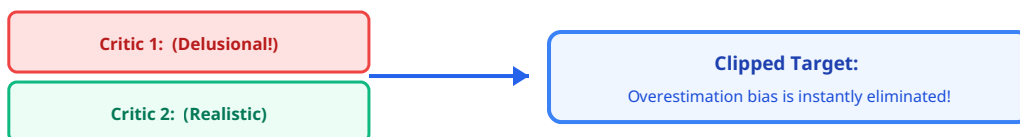
The Polyak Fix: Maintain a separate target network weights $\bar{\theta}$, updated ultra-slowly:

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta} \quad \text{with} \quad \tau = 0.005$$

Every step, the target moves by only **0.5%**. The target moves like molasses ($t_{1/2} \approx 138$ steps), giving the network a crystal-clear, steady bullseye to aim at.

3.3 Super-Weapon 3: Clipped Twin-Q (The Two Skeptical Judges)

$$y = r(s, a) + \gamma \left(\min_{j=1,2} Q_{\bar{\theta}_j}(s', a') - \alpha \log \pi(a' | s') \right)$$



📝 PLAIN ENGLISH TRANSLATION OF CLIPPED TWIN-Q

"Train two independent Critic networks. When computing future value, always pick the **MORE PESSIMISTIC** of the two. If one network hallucinates that a risky blade slam is worth 100 points, the second network grounds it back to reality!"

4. Concrete Numerical Walkthrough: One SAC Bellman Target Update

Let's calculate the exact Bellman target for a transition sampled from the replay buffer:

Step	Variable / Expression	Value	Physical Role
1	Sampled Reward r	+4.0	Blade penetrated 2 mm deeper into fruit.
2	Discount Factor γ	0.99	Patience meter for future rewards.
3	Entropy Temperature α	0.2	Curiosity weighting factor.
4	Critic 1 Target $Q_{\bar{\theta}_1}(s', a')$	32.5	First critic's future prediction.
5	Critic 2 Target $Q_{\bar{\theta}_2}(s', a')$	28.0	Second critic's more conservative prediction.
6	Next Action Log-Prob $\log \pi(a' s')$	-1.5	Entropy bonus: $-\alpha \log \pi = -0.2(-1.5) = +0.3$.
7	Clipped Soft Target y	$4.0 + 0.99(28.0 + 0.3) = \mathbf{32.017}$	Target used to train Critic weights via MSE loss!

5. PyTorch SAC Loss Implementation



COMPLETE PYTORCH SOFT ACTOR-CRITIC LOSS COMPUTATION

```
import torch
import torch.nn as nn

def compute_sac_losses(actor, critic1, critic2, target1, target2, batch, alpha=0.2, gamma=0.99):
    states, actions, rewards, next_states, dones = batch

    # 1. CRITIC LOSS: Compute Bellman Target using Target Twin-Q
    with torch.no_grad():
        next_actions, next_log_probs = actor.sample(next_states)
        q1_target = target1(next_states, next_actions)
        q2_target = target2(next_states, next_actions)
        # Take minimum across twin critics to stop overestimation
        min_q_target = torch.min(q1_target, q2_target) - alpha * next_log_probs
        y = rewards + gamma * min_q_target * (1.0 - dones)

    # Current Q-values
    q1_current = critic1(states, actions)
    q2_current = critic2(states, actions)
    critic_loss = 0.5 * (nn.functional.mse_loss(q1_current, y) + nn.functional.mse_loss(q2_current, y))

    # 2. ACTOR LOSS: Maximize expected Q + Entropy
    new_actions, log_probs = actor.sample(states)
    q_new = torch.min(critic1(states, new_actions), critic2(states, new_actions))
    actor_loss = (alpha * log_probs - q_new).mean()

    return critic_loss, actor_loss
```

6. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	The Hidden Cause	How to Fix It
1. Target Update Too Fast	Setting $\tau = 0.05$ instead of 0.005 moves target networks 10x too quickly. Values diverge to 10^6 within 20,000 steps.	Keep $\tau \in [0.005, 0.01]$. In robotics, slower target updates always mean higher stability.
2. Replay Buffer Starvation	Starting gradient descent before the buffer has collected enough transitions causes the policy to overfit to the first 50 random steps.	Collect at least 10,000 random exploration steps before starting the first neural network update.
3. Action Bounds Violation	Continuous robotic motor torques must be strictly bounded to $[-1, +1]$. Forgetting <code>torch.tanh()</code> causes motor commands to spike to infinity.	Always use Tanh-squashed Gaussian distributions with proper Jacobian log-prob correction.

CS285 Lecture 8 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An